Note: The REST API example was earlier published on my blog at *http://echorand.me/2012/01/27/picloud-and-rest-api-with-c-client/*

That is, you can have a function written in Python on the cloud, and you can have a REST client written in any other language calling this function. Let us jump straight into an example from PiCloud's documentation on this topic *(http://docs.picloud.com/rest.html#publishing)*. Create a file *publish_square.py* with the following function:

```
# Refer: http://docs.picloud.com/rest.html
import cloud
def square(x):
    """Returns square of a number"""
    print 'Squaring %d' % x
    return x*x
```

Now, publish this function using the *cloud.rest.publish* method:

```
>>> uri=cloud.rest.publish(square, "square_func")
>>> print uri
https://api.picloud.com/r/3222/square_func
```

The above statement makes the function square accessible via the REST API, and returns a URI for the same. You can now access this by making an appropriate call to the API using a command-line tool such as *curl*, or by using any other programming language. In this article, let us examine a C client for this. We used *libcurl's C* interface (*http://curl.haxx.se/libcurl/c/*) to replicate the *curl* statements provided in the PiCloud documentation. The client is written in two parts: client1.c and client2.c, which you can download from *bitbucket.org*—see the last link under *Resources*, at the end of this article. *Client1.c* invokes the published function, hence gets the job ID. *Client2.c* then uses this job ID to get the result. Compile and run the programs, as follows:

```
$ gcc -o client1 client1.c -lcurl
$ gcc -o client2 client2.c -lcurl

$ ./client1
{"jid": 57, "version": "0.1"}
Result of Operation:: 0
$ ./client2 57
{"version": "0.1", "result": 25}
Result of Operation:: 0
```

For detailed information on publishing functions, please refer to the PiCloud documentation on this topic—*http://docs.picloud.com/rest.html*.

Running an Evolutionary Algorithm framework in the cloud

Our examples in this article have all been rather simple, since they were meant to clear the concepts. In a more real-life use of PiCloud, let us take a look at a simple way of using PiCloud in the domain of Evolutionary Algorithms. There are a number of ways in which an Evolutionary Algorithm can be parallelised. One of the easiest things to do is to run parallel instances of the algorithm with different initial random seeds. (This is a common practice in the research community, since starting with different random seeds allows you to test the robustness of a new algorithm, or the correct implementation of an existing one.)

In a blog post, I detailed the first exercise that I tried with *Pyevolve* (a Python library for Evolutionary Algorithms) + PiCloud—to run Pyevolve's Genetic Algorithm implementation with 10 different initial seeds on PiCloud, all running in parallel. Here is the link to it: *http://echorand.me/2012/01/26/picloud-pyevolve-evolutionary-algorithms-in-the-cloud/*

PiCloud Web interface

Once you log in to your account on PiCloud (*https://www.picloud.com/accounts/*), you should see the PiCloud Web control panel, where you can find the current running jobs, API keys, published functions and much more.

In this article, we have taken a very basic tour of PiCloud, and looked at most of its important features. We haven't, however, looked at features like the ability to run *cron jobs*, *using different computing powers* and *using environments*. You are requested to consult the official documentation, which I have listed below, for these and more detailed discussions and examples on the topics we have discussed in this article. END

Resources

- PiCloud Documentation Home: *http://docs.picloud.com/*
- API Documentation: *http://docs.picloud.com/moduledoc.html*
- PiCloud REST API: *http://docs.picloud.com/rest.html*
- PiCloud blog: *http://blog.picloud.com/*
- Article code: *https://bitbucket.org/amitksaha/articles_code/src/eb37714f3a2e/picloud_article*

By: Amit Saha

The author is currently doing his PhD in the area of Evolutionary Algorithms and Optimisation. Like his random echoes would show *(http://echorand.me/)*, he has been writing on various Linux and Open Source technologies over the last few years. Overall, he loves playing around with a bit of this and a bit of that. He welcomes comments on this article and beyond at *amitsaha.in@gmail.com*.